

GUIDE TECHNIQUE

Créer et Comprendre un SDK

avec Jetpack Compose (Android)

Définition, utilité, architecture et étapes de création d'un SDK Android moderne

Juillet 2026

Table des matières

Table des matières	2
1. Introduction	3
2. Qu'est-ce qu'un SDK ?	3
2.1 Définition	3
2.2 Composants typiques d'un SDK	3
2.3 SDK vs Librairie vs API	3
3. Pourquoi créer un SDK ?	4
3.1 Avantages pour l'éditeur	4
3.2 Avantages pour l'équipe qui l'intègre	4
4. Introduction à Jetpack Compose	4
4.1 Concepts clés	4
4.2 Exemple minimal	4
5. Pourquoi créer un SDK avec Jetpack Compose ?	5
5.1 Bénéfices spécifiques	5
6. Architecture d'un SDK Compose	5
6.1 Les quatre couches	5
6.2 Exemple de structure de module Gradle	5
7. Étapes de création d'un SDK Compose	6
Étape 1 — Créer un module Android Library	6
Étape 2 — Concevoir une API publique claire	6
Étape 3 — Créer un thème personnalisable	6
Étape 4 — Documenter le SDK	7
Étape 5 — Tester	7
Étape 6 — Versionner et publier	7
8. Bonnes pratiques	7
9. Conclusion	8

1. Introduction

Ce document a pour objectif d'expliquer, de façon claire et progressive, ce qu'est un SDK (Software Development Kit), pourquoi il est essentiel dans le développement mobile, et comment en concevoir un moderne à l'aide de Jetpack Compose, le framework déclaratif d'Android pour la création d'interfaces utilisateur.

Il s'adresse aux développeurs Android souhaitant transformer une librairie interne, un composant réutilisable ou une brique fonctionnelle (paiement, authentification, carte bancaire, scanner, etc.) en un SDK distribuable, documenté et maintenable.

À retenir

Un SDK n'est pas qu'un simple fichier .aar : c'est un produit à part entière, avec une API publique stable, une documentation, un versioning et un canal de distribution.

2. Qu'est-ce qu'un SDK ?

2.1 Définition

Un SDK (Software Development Kit) est un ensemble d'outils, de librairies, de code, de documentation et d'exemples permettant à un développeur d'intégrer facilement une fonctionnalité ou un service tiers dans son application, sans avoir à en réécrire la logique.

Sur Android, un SDK est le plus souvent distribué sous la forme d'un ou plusieurs fichiers .aar (Android Archive) ou publié en tant que dépendance Gradle via un dépôt Maven (Maven Central, Google Maven, ou un dépôt privé de type Nexus ou Artifactory).

2.2 Composants typiques d'un SDK

- Code compilé (classes, ressources) — souvent packagé en .aar ou .jar
- API publique — les classes, fonctions et interfaces exposées aux développeurs
- Documentation — README, KDoc/Javadoc, guides d'intégration
- Exemples d'utilisation — application de démonstration (sample app)
- Fichier de configuration — build.gradle, proguard-rules, AndroidManifest
- Système de versioning — SemVer (ex. 1.5.8) pour gérer la compatibilité

2.3 SDK vs Librairie vs API

Terme	Définition	Exemple
Librairie	Ensemble de code réutilisable, souvent interne au projet	Module Gradle local ':core-utils'
API	Contrat d'échange entre systèmes (souvent réseau/HTTP)	API REST de paiement
SDK	Package complet incluant code, doc et outils pour intégrer une API ou un service	SDK de carte bancaire idMobile

3. Pourquoi créer un SDK ?

Créer un SDK présente de nombreux avantages, tant pour l'éditeur (l'entreprise qui le distribue) que pour les équipes qui le consomment.

3.1 Avantages pour l'éditeur

- Réutilisabilité : une seule base de code partagée entre plusieurs applications
- Cohérence : toutes les apps utilisent la même logique métier, les mêmes règles de sécurité
- Maintenance centralisée : un correctif est publié une fois, puis mis à jour partout
- Contrôle de version : chaque application choisit la version compatible du SDK
- Monétisation possible si le SDK est distribué à des clients externes

3.2 Avantages pour l'équipe qui l'intègre

- Gain de temps : pas besoin de redévelopper une fonctionnalité complexe (ex. lecteur de carte NFC, scoring biométrique)
- Fiabilité : le SDK est testé et déjà utilisé en production ailleurs
- Documentation claire et API stable, ce qui réduit les erreurs d'intégration
- Mises à jour simples via Gradle (changement de numéro de version)

Exemple concret

Dans une application bancaire mobile, des SDK tiers gèrent souvent la lecture de carte à puce (ex. idMobile), la biométrie (SmartSDK) ou le paiement sans contact — autant de briques que l'on ne réécrit pas soi-même.

4. Introduction à Jetpack Compose

Jetpack Compose est le toolkit moderne de Google pour construire des interfaces utilisateur natives sur Android. Il remplace progressivement le système de vues XML (View/XML) par une approche déclarative et fonctionnelle, écrite entièrement en Kotlin.

4.1 Concepts clés

- Fonctions `@Composable` : blocs de fonctions qui décrivent l'UI en fonction d'un état
- État (State) : les composables se redessinent automatiquement (recomposition) quand l'état change
- Unidirectional Data Flow : l'état descend, les événements remontent
- Modifier : chaîne d'API permettant de personnaliser taille, style, comportement d'un composant
- Thème (MaterialTheme) : gestion centralisée des couleurs, typographies et formes

4.2 Exemple minimal

```
@Composable
fun WelcomeCard(userName: String, onClick: () -> Unit) {
    Card(
        modifier = Modifier
            .padding(16.dp)
            .clickable { onClick() }
    ) {
        Text(
```

```

        text = "Bonjour, $userName !",
        style = MaterialTheme.typography.titleMedium
    )
}

```

5. Pourquoi créer un SDK avec Jetpack Compose ?

De plus en plus de SDK Android — notamment ceux qui exposent des écrans complets (ex. formulaire de paiement, écran de scan de carte, écran d'authentification biométrique) — sont construits directement avec Jetpack Compose plutôt qu'en XML.

5.1 Bénéfices spécifiques

Bénéfice	Explication	Impact
Intégration simplifiée	L'application hôte insère un simple <code>@Composable</code> , sans gérer de <code>Fragment/Activity</code>	Moins de code côté client
Personnalisation facile	Le thème (couleurs, polices) du SDK peut hériter du <code>MaterialTheme</code> de l'app hôte	UI cohérente avec la marque
Moins de code XML	Pas de <code>layout.xml</code> à maintenir en parallèle du code Kotlin	Maintenance réduite
Aperçu instantané	<code>@Preview</code> permet de visualiser les écrans du SDK sans lancer l'app	Développement plus rapide
Interopérabilité	Compose peut cohabiter avec des Views XML existantes (<code>ComposeView</code>)	Migration progressive possible

6. Architecture d'un SDK Compose

Un SDK bien conçu sépare clairement les responsabilités. On distingue généralement quatre couches :

6.1 Les quatre couches

1. Couche Core (logique métier) — ne dépend pas de Compose ; contient les modèles, le réseau, la sécurité
2. Couche UI (Compose) — les écrans, composants et thèmes exposés au développeur intégrateur
3. Couche API publique (Facade) — point d'entrée unique et stable du SDK (ex. `SdkManager.init(...)`)
4. Couche interne (Internal) — classes marquées `internal`, invisibles depuis l'application hôte

6.2 Exemple de structure de module Gradle

```

mon-sdk/
├── build.gradle.kts
├── src/main/java/com/exemple/sdk/
│   ├── core/           -> logique métier, réseau, modèles
│   ├── ui/
│   │   └── components/ -> composants Compose réutilisables

```

├── screens/	-> écrans complets (@Composable)
├── theme/	-> Color.kt, Type.kt, Theme.kt
├── SdkConfig.kt	-> configuration exposée
├── SdkManager.kt	-> point d'entrée public (Facade)
└── proguard-rules.pro	

7. Étapes de création d'un SDK Compose

Étape 1 — Créer un module Android Library

Dans Android Studio : File > New > New Module > Android Library. Ce module aura le plugin `com.android.library` (et non `com.android.application`) puisqu'il ne produit pas d'APK mais un `.aar`.

```
// build.gradle.kts du module SDK
plugins {
    id("com.android.library")
    id("org.jetbrains.kotlin.android")
}

android {
    namespace = "com.exemple.mysdk"
    compileSdk = 34
    buildFeatures { compose = true }
    composeOptions {
        kotlinCompilerExtensionVersion = "1.5.8"
    }
}
```

Étape 2 — Concevoir une API publique claire

L'idée est d'exposer le minimum nécessaire et de cacher les détails d'implémentation avec le mot-clé `internal`. Une bonne API publique tient généralement en quelques classes et fonctions.

```
object MySdk {
    fun init(context: Context, apiKey: String) { /* ... */ }

    @Composable
    fun PaymentScreen(
        amount: Double,
        onSuccess: () -> Unit,
        onError: (SdkError) -> Unit
    ) {
        // écran Compose exposé aux intégrateurs
    }
}
```

Étape 3 — Créer un thème personnalisable

Un SDK Compose doit permettre à l'application hôte d'adapter les couleurs et polices, tout en fournissant des valeurs par défaut cohérentes.

```
@Composable
```

```
fun MySdkTheme(
    colors: MySdkColors = MySdkDefaults.colors,
    content: @Composable () -> Unit
) {
    MaterialTheme(
        colorScheme = colors.toMaterialScheme(),
        content = content
    )
}
```

Étape 4 — Documenter le SDK

- Ajouter des commentaires KDoc sur chaque fonction publique
- Rédiger un README.md avec un exemple d'intégration en 5 lignes maximum
- Fournir une application d'exemple (sample-app) dans le même repository Git

Étape 5 — Tester

- Tests unitaires sur la couche core (JUnit, MockK)
- Tests de composants Compose avec ComposeTestRule et createComposeRule()
- Tests d'intégration dans une app de démonstration réelle

Étape 6 — Versionner et publier

Le SDK est ensuite publié comme dépendance Maven, généralement sur un dépôt privé d'entreprise (Nexus, Artifactory) ou public (Maven Central). Cela permet à toute application de l'ajouter simplement via Gradle.

```
// build.gradle.kts du module SDK
publishing {
    publications {
        create<MavenPublication>("release") {
            groupId = "com.exemple"
            artifactId = "mysdk"
            version = "1.5.8"
        }
    }
    repositories {
        maven { url = uri("https://nexus.exemple.fr/repository/maven-releases/") }
    }
}
```

Côté application cliente, l'intégration se résume alors à une simple ligne dans le fichier de dépendances :

```
dependencies {
    implementation("com.exemple:mysdk:1.5.8")
}
```

8. Bonnes pratiques

Bonne pratique	Pourquoi
Suivre le Semantic Versioning (MAJOR.MINOR.PATCH)	Permet aux intégrateurs de savoir si une mise à jour casse la compatibilité
Garder l'API publique minimale	Réduit le risque de breaking change et facilite la maintenance
Isoler la logique métier de l'UI Compose	Facilite les tests et une éventuelle réutilisation hors Compose
Fournir un thème par défaut mais personnalisable	Le SDK s'intègre visuellement dans n'importe quelle app
Documenter chaque breaking change dans un CHANGELOG	Transparence vis-à-vis des équipes intégratrices
Utiliser ProGuard/R8 rules dédiées	Évite que l'obfuscation ne casse le SDK chez le client

9. Conclusion

Un SDK est bien plus qu'un simple fichier binaire : c'est un produit logiciel à part entière, pensé pour être intégré facilement, durablement et de façon cohérente dans des applications tierces. Jetpack Compose, grâce à son approche déclarative, simplifie considérablement la création de SDK exposant des interfaces utilisateur : moins de code, une meilleure personnalisation via les thèmes, et une intégration réduite à quelques lignes pour les équipes qui l'utilisent.

En résumé, un bon SDK Compose repose sur trois piliers : une API publique claire et stable, une séparation nette entre logique métier et interface, et une documentation soignée qui accompagne chaque version publiée.

Pour aller plus loin

Documentation officielle : developer.android.com/jetpack/compose — et pour la publication de bibliothèques : developer.android.com/studio/publish-library